ToyDB: complete project summary

Parts 1, 2, 3A and 3B | 3 November 2026

Team: B24CS1087 and B24CS1113

Project status

All three assignment parts are implemented: PF page buffering, slotted-page record storage with a fixed-length comparison, and three methods for building the Student roll-number B+ tree index. The final submission archive has been built and tested after extraction.

Reading guide

Section	Combined pages	Content
Part 1	2-6	Original ten-step PF implementation, global replacement decision, tests, graphs and handoff.
Part 2	7-10	Original slotted/fixed record layouts, six design decisions, correctness and space/I/O experiments.
Part 3A	11-14	Original Methods 1-2, AM scan fixes, validation, preliminary measurements and bulk-loader handoff.
Part 3B / final comparison	15-18	Completed bulk loader, final validation, current three-method results, limitations and graphs.

How historical summaries should be read

The original summaries are preserved in full. Their statements that later parts are pending describe the project at the time they were written. Those pending tasks have since been completed. Earlier timings are historical runs and may differ from the final comparison. The Part 3B section and current outputs/index-construction/comparison.md contain the final three-method measurements.

Final checks and the retained design decision

All 26 PF/slotted-page regression cases and all five indexing test groups passed. AddressSanitizer and UBSan checks passed with leak detection disabled because this container cannot run LeakSanitizer under tracing. A clean archive extraction compiled, passed tests, and ran all methods and graph generation.

The agreed first-iteration policy remains a **shared global buffer pool with a replacement policy selected per open file**. The requesting file selects LRU/MRU; the victim may belong to any file. Local replacement remains a possible later iteration and is not part of this submission.

Submission: B24CS1087-B24CS1113.tar.gz. Source and result locations are documented in the root README. Git commit/push and classroom submission are separate from the implementation and package checks.

ToyDB: page buffering summary

Part 1 implementation and handoff | 2 November 2026

This document summarizes the completed ten-step PF page-buffering work. It records implementation details, validation, experiment results and the agreed design choice. Part 2 (slotted pages) is the next task for the teammate; part 3 (index construction) remains pending.

1. Compile and test the supplied code

Rebuilt the original PF source with GCC 13.3 using make -B tests. Both supplied programs, testhash and testpf, completed successfully. No source compatibility fixes were needed for that baseline. Legacy C warnings remain; passing tests do not imply the code has been fully modernized. Tests run in scratch directories to preserve the supplied files.

2. Understand the existing page flow

Open a file and obtain a PF descriptor. Request a page using its descriptor and page number. The buffer manager checks its hash table; a hit reuses RAM, while a miss obtains a frame and reads the page. The page remains fixed while in use. Unfixing releases it for possible replacement but leaves it cached. Dirty victims are saved before reuse, and closing a file flushes its dirty pages after all pages are unfixed.

The disk file contains a file header and fixed-size stored pages. RAM contains the open-file table, buffer frames and lookup structures. A PF descriptor indexes the open-file table; the entry contains the underlying OS descriptor. A buffered page is identified by the pair (PF descriptor, page number).

3. Make the buffer size configurable

Added PF_InitWithBufferSize(count), which validates the requested positive count, rejects initialization while files are open or after frames have been allocated, sets the capacity and calls the existing PF_Init. buf.c now checks a runtime capacity instead of the fixed limit. Frames are allocated on demand. Legacy callers retain the default of 20 frames; live resizing is unsupported.

4. Add replacement-policy choice when opening a file

Added PF_OpenFileWithPolicy(name, policy), with PF_POLICY_LRU and PF_POLICY_MRU. Each open-file entry stores its selected policy. PF_GetFilePolicy(fd) retrieves it. Invalid policies are rejected before opening the file. Existing PF_OpenFile(name) selects LRU, preserving existing callers. The pool remains shared across files.

Implementation: replacement, updates and counters

5. Implement MRU and verify LRU

PFbufInternalAlloc now receives the requesting file descriptor and retrieves its selected policy. LRU searches from the least-recent end of the global list; MRU searches from the most-recent end. Both skip fixed pages. If every frame is fixed, the request returns PFE_NOBUF. Dirty victims are saved using their own file descriptor, even when another file triggered eviction.

Recency follows the supplied implementation: new frames enter at the head; unfixing a page or calling PFbufUsed moves it to the head. Fixed pages cannot be victims regardless of list position.

6. Add an explicit operation to mark a page dirty

Added PF_MarkDirty(fd, pageNum), which validates the descriptor and page number and reuses PFbufUsed to mark a fixed page dirty and most recently used. It keeps the page fixed and does not immediately write it. A later PF_UnfixPage(..., FALSE) preserves dirty status. PF_UnfixPage(..., TRUE) remains supported. Dirty status is retained until successful writeback.

7. Add statistics

Added the pool-wide PF_Stats structure and PF_ResetStats(), PF_GetStats() and PF_PrintStats(). Resetting counters does not clear the buffer or flush it.

Measurement	Implemented meaning
Logical reads	Successful buffer fetches, including internal fetches and scans of free pages.
Logical writes	Successful update notifications through dirty unfixing or explicit dirty marking.
Physical reads / writes	Successful full-page transfers to/from the operating system; writes include eviction and final close flushes.
Buffer hits / misses	Successful fetches classified by whether the page was already cached.

Counts are operations, not distinct pages. File-header I/O, failed operations and fresh frame/page allocation itself are excluded. The OS may cache transfers, so physical I/O here does not mean hardware disk activity. A write query fetches a page and reports one update: one logical read plus one logical write. Do not report the same update through both dirty APIs unless two notifications are intended.

8. Check correctness with small examples

All 12 test cases passed again while preparing this handoff: the two original tests, teststats, five capacities (1, 2, 5, 20, 32), and LRU/MRU variants of testreplacement and testdirty. Tests verify exact victims, hit recency, cross-file policy selection, pinned-page protection, full-pool errors, dirty persistence through eviction/reopening, invalid inputs and exact counter totals/resetting.

Experiments and the agreed design decision

9. Run read/write experiments and produce graphs

Added benchmark.c and a configurable Python experiment runner. Defaults: 20 buffer frames, a 100-page file, 10,000 queries per run, sequential and random access, both policies, five read/write mixtures (100/0, 75/25, 50/50, 25/75, 0/100), and three repetitions: 60 trials total.

Each trial starts with an empty ToyDB pool after file creation and close. Setup is excluded. The same seeded page trace is used for both policies and all mixtures within a repetition. Writes are deterministically interspersed and toggle one byte; each reports dirty status once. Counts and monotonic elapsed time include the final dirty flush. CSV files retain each run and means; graph X-axes show read/write mixtures and Y-axes show page-operation statistics.

Default sequential result	LRU	MRU
Physical reads (each mixture)	10,000	8,100
Physical writes (50% writes)	5,000	4,050
Buffer hits	0	1,900

MRU reduces physical reads by 19% for this repeated-scan workload. Random-access results are similar between policies and vary by seed. These synthetic page workloads do not yet use student records and do not establish that MRU is always faster. OS caching and very short run times limit timing conclusions.

10. Document how to run it

Added the root README and PF-specific README, reproducible test runner, experiment configuration and graph runner. Experiment settings live in experiments.json so capacities, workloads, policies and repetitions can change without editing the C source. The exact run configuration is saved with the results. Build and execution commands are on the handoff page.

Design decision: global victims, file-specific policy

Question: The assignment says the replacement policy should be selectable per file, but does not specify whether replacement must be local to a file or global.

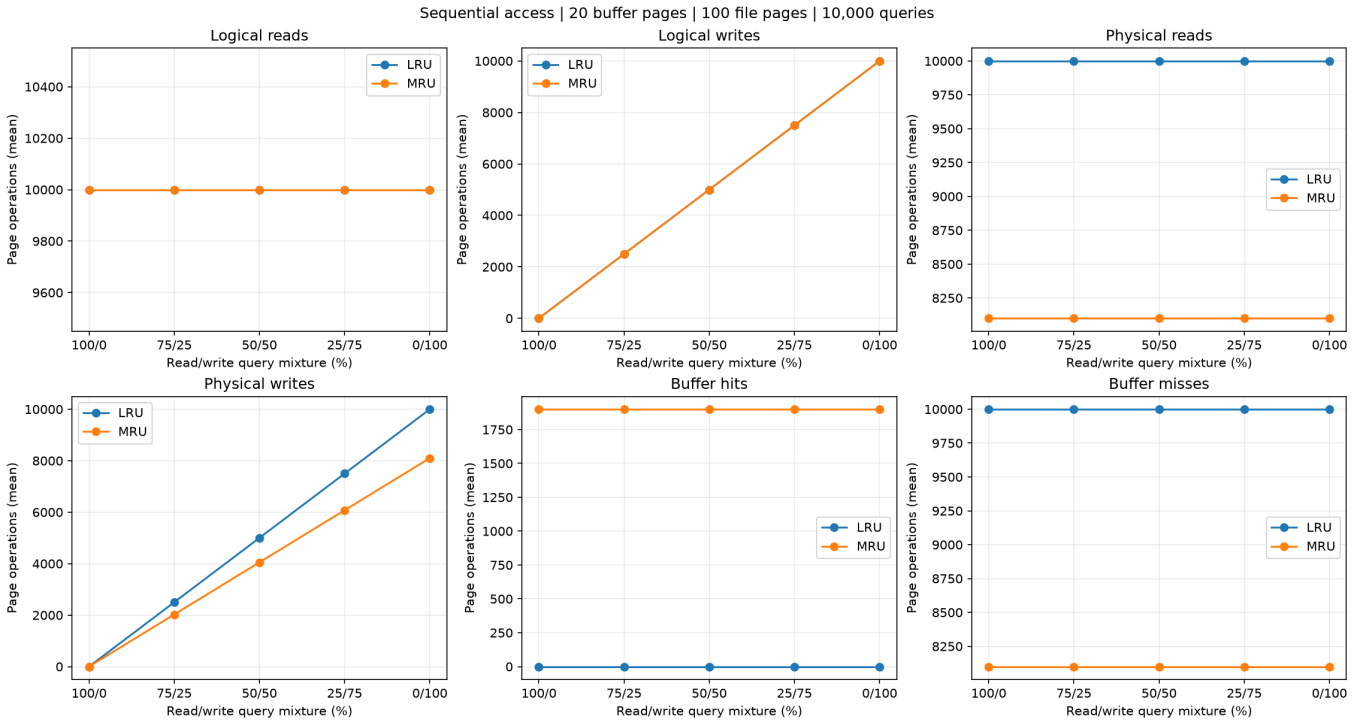
Choice 1 - local replacement: Evict only a page belonging to the requesting file. This requires allocation rules, such as how many frames each file receives. Fixed per-file allocations can restrict how many files can actively use the pool. A rule is also needed when all allocated pages are fixed or a new file has no frame of its own to replace.

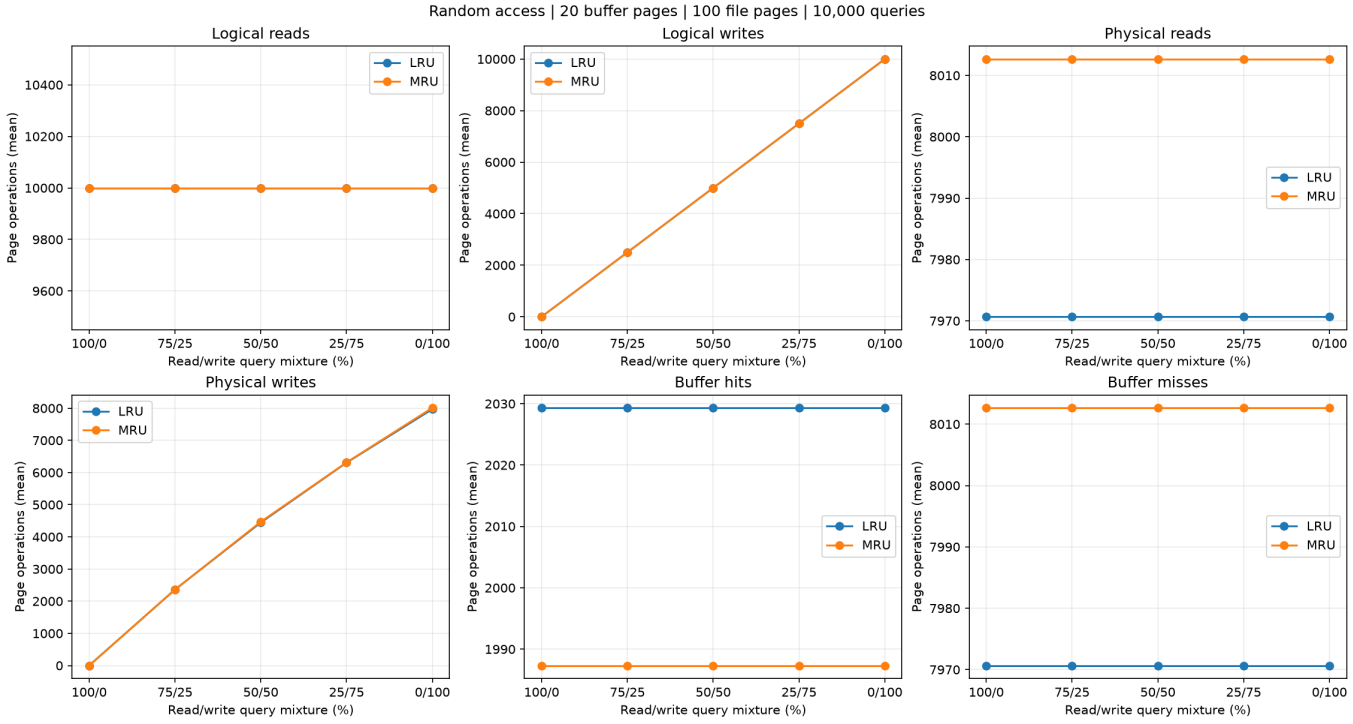
Choice 2 - global replacement: Use the requesting file's policy to evict any eligible global page. This is simpler and avoids per-file frame reservations.

Agreed decision: Use choice 2 for the first iteration. Complete all three assignment parts before revisiting local replacement. This is a documented interpretation of ambiguous requirements, not an explicit requirement in the assignment. The same decision is recorded in outputs/design-decisions/toydb-design-decisions.xlsx.

Experiment graphs

Each panel shows the mean of three repetitions. Lines overlap where both policies have identical counts. Configuration: 20 frames, 100 file pages, 10,000 queries per run.





Source outputs: outputs/page-buffering/results.csv, summary.csv and config.json. Elapsed times are available in the CSV files.

Teammate handoff: starting part 2

Build the slotted-page record layer on top of the PF interfaces. Store page headers, a slot directory and variable-length record bytes inside the PF-provided data area (PF_PAGE_SIZE = 4096 bytes). Do not alter the PF disk-page header. Decide the record identifier format (typically page number plus slot number) and coordinate it with the later roll-number index.

For insertion/deletion: fetch or allocate a page, modify its bytes, mark it dirty and unfix it on every completed path. For scanning: fetch each page, inspect its live slots and unfix it before moving on. Do not retain page pointers after unfixing. Avoid keeping many pages fixed; the global pool can otherwise run out of eligible frames.

```
/* Illustrative call sequence; check every return value. */
PF_InitWithBufferSize(20);
fd = PF_OpenFileWithPolicy("students.pf", PF_POLICY_LRU);
PF_GetThisPage(fd, pageNum, &pageBuf);
/* Read or update the slotted-page bytes in pageBuf. */
PF_MarkDirty(fd, pageNum);          /* only after updates */
PF_UnfixPage(fd, pageNum, FALSE);
PF_CloseFile(fd);
```

Part 2 still needs insertion, deletion, sequential scanning, student.txt loading, space-utilization metrics and a table comparing variable-length storage with fixed-length storage at different maximum record lengths. Part 3 still needs the index-construction comparison and sorted bulk loading. AM-layer compilation/tests have not been validated in this PF task.

Build, test and reproduce from the repository root

```
make -B -C toydb/pflayer tests benchmark
python3 toydb/pflayer/run_tests.py

# Python environment for graphs (matplotlib required):
python3 -m venv /tmp/toydb-venv
/tmp/toydb-venv/bin/pip install matplotlib
/tmp/toydb-venv/bin/python toydb/pflayer/run_experiments.py
```

Edit toydb/pflayer/experiments.json before running experiments. Optional runner arguments: --config path.json --output directory. The current machine also has matplotlib under /tmp/sailor-plot-deps; that temporary installation is not part of a Git checkout. Rebuild source after pulling; do not rely on supplied or regenerated binaries.

File map for review

Category	Files (relative to toydb/pflayer unless noted)
Modified implementation	pf.c, pf.h, pftypes.h, buf.c; Makefile builds all tests and benchmark.
Added tests and runner	testcapacity.c, testreplacement.c, testdirty.c, teststats.c, run_tests.py. Original testpf.c and testhash.c remain in use.
Added experiments	benchmark.c, experiments.json, run_experiments.py.
Documentation / outputs	Root README.md; PF README.md; outputs/page-buffering/ CSVs, config and graphs; decision spreadsheet; root summary.pdf.

Source basis: assignment Objective and submission instructions; supplied pf.ps and PF implementation; current README files, PF headers, test runner and saved experiment configuration/results. No commit or push was performed as part of preparing this summary.

ToyDB: slotted pages summary

Part 2 implementation and handoff | 2 November 2026

This document summarizes the completed part-2 work: a slotted-page layer for variable-length records on top of the PF layer, a separate fixed-length (static) layer used as the baseline, their tests, and the space and I/O experiments on the 17,813 student records. Part 3 (index construction on roll-no) is next.

1. What was built

File (toydb/splayer/)	Purpose
`slotted.h/.c`	Slotted-page layer: create/open/close, insert, delete, get by RID, sequential scan, space statistics
`fixedrec.h/.c`	Fixed-length layer with the same operations; every record occupies exactly L bytes
`recfile.h/.c`	Shared definitions only: RID packing, error codes, search modes, the space-statistics structure
`studentdata.h/.c`	Reads `data/student.txt` into raw records (title and blank lines skipped)
`testslotted.c, testfixed.c, testload.c`	C tests; `run_tests.py` builds and runs them all and writes a report
`spbench.c, run_experiments.py, experiments.json`	Experiment driver, runner and configuration

No PF or AM source file was modified. Both layers use only the PF interface (`PF_AllocPage`, `PF_GetThisPage`, `PF_GetNextPage`, `PF_MarkDirty`, `PF_UnfixPage`, `PF_OpenFileWithPolicy`).

2. Slotted page layout

The whole 4096-byte PF data area is used; the PF layer keeps its own page header outside it. All fields are 16-bit integers at fixed offsets - no pointers are stored on disk.

```
offset 0  magic 0x5350 | 2 numSlots | 4 liveCount | 6 freeEnd
offset 8  slot directory: numSlots x {offset, length}      (empty slot: offset 0)

[header 8B][slot 0][slot 1]...[slot n-1]    free space    [rec]...[rec][rec]
                              ^freeStart = 8 + 4n       ^freeEnd      ^4096
```

The directory grows forward and records grow backward. Because every delete compacts the page, records always fill exactly `[freeEnd, 4096)`; each fetched page is validated against this. Largest record: 4084 bytes. A record id is `RID = page × 65536 + slot` - one non-negative `int`, the type `AM_InsertEntry` expects (pages 0..32767, so the largest RID is exactly INT_MAX).

3. Operations

- **Insert**: first page with enough contiguous room (reusing an empty slot if the page has one, otherwise needing 4 more bytes for a new slot); a new page only if none fits. Mark dirty once, unfix.
- **Delete**: shift the records below the deleted one up by its length, fix their offsets, mark the slot empty, drop empty slots at the end of the directory. Slot numbers of live records never change.
- **Scan**: copies each page into the scan handle and unfixes it at once - one logical read per page and no page held fixed between calls, so even a 1-frame buffer pool works.

The six agreed design decisions

Question	Options considered	Agreed decision and consequence
1. Record representation	A raw text line; B fields with length prefixes	**A.** Each student.txt line stored unchanged; fields split on ";" when needed. Lets every record be verified byte-for-byte.
2. RIDs and slot reuse	A reuse deleted slots; B never reuse slot numbers	**A.** RID = page × 65536 + slot. Lowest empty slot reused; trailing empty slots trimmed. **Part 3 rule:** delete a student's index entry before its slot can be reused, or the entry would point at a different record.
3. Records longer than L (fixed layout)	A truncate; B reject; C only test L ≥ longest	**C with rejection reporting.** L ∈ {100, 108, 128, 160, 200, 256}; longest record is 108 bytes. Rejected records are counted per L, never truncated.
4. Utilization formula	A record bytes / (pages × 4096); B exclude page overhead	**A** for both layouts. Every page byte is record data, metadata, padding or free; the four always sum to pages × 4096 (checked by the tests).
5. Page for an insert	A last page; B first fit; C on-disk free-space map	**B** for the load and later inserts. Default search keeps each page's free space in memory (rebuilt on open) and fetches only the chosen page; a literal page-by-page scan is kept for comparison. Both choose identical pages.
6. When to compact	A on every delete; B lazily on insert	**A.** Free space is always one contiguous region, so an insert only compares sizes; RIDs stay valid.

Smaller implementation choices (flagged for review)

- The fixed layout pads with `'\0'` and derives a record's length from the first pad byte, so it stores no per-slot length (purely static); records containing `'\0'` are rejected.
- Pages are never released after deletes: utilization falls in proportion to deletions and re-inserted records refill the same pages.
- The supplied K&R PF code does not compile with GCC 14+ defaults (GCC 15 also defaults to C23). The Makefile builds it with `-std=gnu17 -fpermissive` instead of editing it. The part-1 build command needs the same flags; the root README is updated.

Full options, results and evidence: `outputs/design-decisions/part2-design-decisions.md` and the "Part 2 slotted pages" sheet of `toydb-design-decisions.xlsx`.

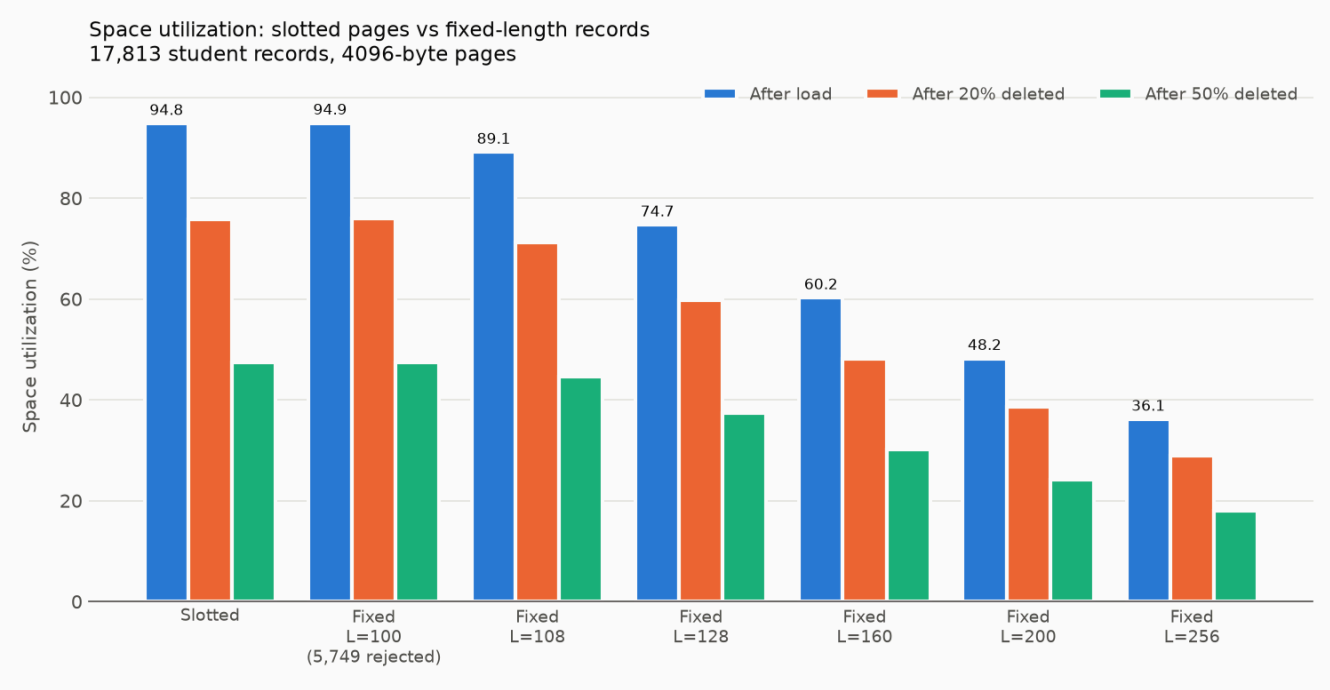
Tests: 26 of 26 cases pass

Program (cases)	What it verifies
`testslotted` (6)	RID boundaries; insert/get/error codes; max-size record; exact first-fit placements; compaction leaves one free region and other records intact; slot reuse; directory trimming; one logical read + one logical write per insert/delete; persistence after reopen; rejection of non-slotted pages; 4,000 random operations against an in-memory model with both search modes producing identical RIDs
`testfixed` (6)	Slots per page for each L; oversized and `'\0'` rejection; slot reuse; I/O counts; padding accounting; L mismatch and slotted files refused on open; 4,000 random operations
`testload` (2)	All 17,813 student records loaded into slotted and fixed (L = 100, 108, 128, 256), read back by RID and by scan, byte-for-byte; rejects exactly the records longer than L
part-1 PF tests (12)	Re-run after a genuine rebuild with the flags above (`run_tests.py --with-pf`)
Buffer sizes	Unit tests run with 1-, 2- and 20-frame pools in both search modes; the 1-frame runs prove no operation leaves a page fixed. Report: `outputs/slotted-pages/test-results.txt`

Space utilization: slotted vs static

17,813 records, 1,759,011 bytes, longest 108 bytes. 4096-byte pages, 20-page LRU pool, first-fit insertion.

Layout	L	Stored	Rejected	Pages	Rec/page	Metadata B	Padding B	Free B	Utilization
Slotted	-	**17,813**	**0**	**453**	**39.3**	**74,876**	**0**	**21,601**	**94.80%**
Fixed	100	12,064	5,749	302	39.9	3,926	57,816	1,600	94.88%
Fixed	108	17,813	0	482	37.0	6,266	206,727	2,268	89.10%
Fixed	128	17,813	0	575	31.0	6,900	587,753	1,536	74.69%
Fixed	160	17,813	0	713	25.0	8,556	1,150,961	1,920	60.23%
Fixed	200	17,813	0	891	20.0	9,801	1,879,324	1,400	48.20%
Fixed	256	17,813	0	1,188	15.0	11,880	3,093,365	1,792	36.15%

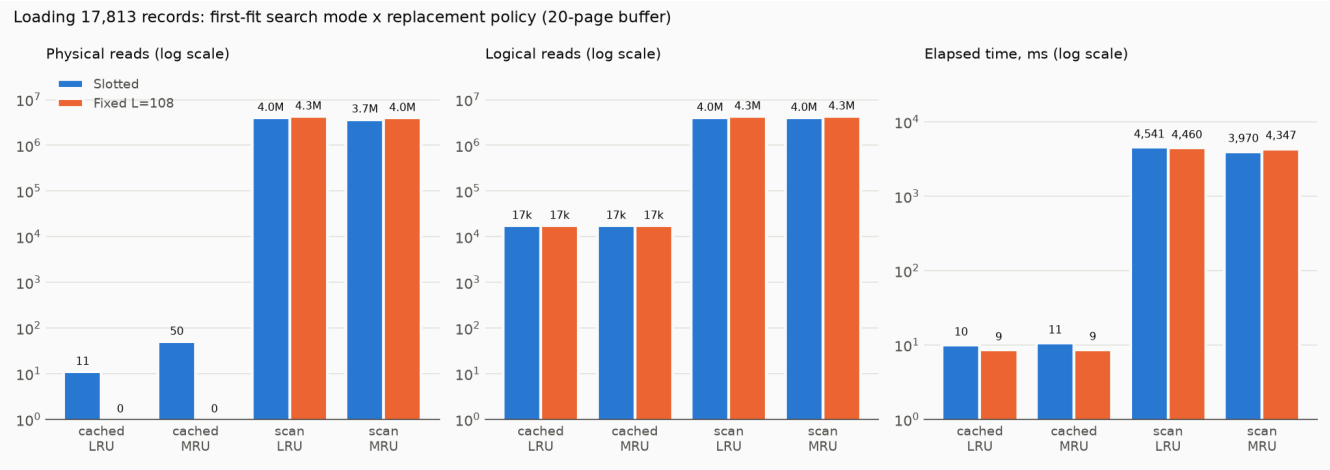


- **Slotted pages beat every fixed layout that stores all records.** Slotted pays 4 bytes of slot directory per record (74,876 B in total); fixed L = 108 pays 206,727 B of padding, and padding grows linearly with L.
- Fixed L = 100 matches slotted utilization only by rejecting 32% of the students.
- After deleting 20% / 50%, slotted utilization is 75.84% / 47.40% (fixed L = 108: 71.28% / 44.55%). Re-inserting the same records restores the after-load figure exactly in every layout; the page count never changes.

I/O cost and handoff

Load phase, 20-page pool; every modifying phase ends with a close so all dirty pages are written inside it. Counts are identical over 3 repetitions; times are means.

Layout	Search	Policy	Logical reads	Physical reads	Physical writes	Time (ms)
Slotted	cached	LRU	17,360	11	464	10
Slotted	scan	LRU	4,011,349	4,003,039	17,038	4,568
Slotted	scan	MRU	4,011,349	3,677,408	14,704	4,195
Fixed 108	cached	LRU	17,331	0	482	9
Fixed 108	scan	LRU	4,296,307	4,288,537	17,093	4,624
Fixed 108	scan	MRU	4,296,307	3,961,885	14,878	4,098



- The literal first-fit scan re-reads the full pages on every insert: about 4 million reads to load the file, roughly 450× slower than the cached search, which places every record identically.
- Under that repeated-scan pattern MRU saves about 8% of physical reads over LRU - the part-1 effect on a real workload. A sequential scan costs one read per page (453 slotted, 482 fixed L = 108).
- Times use a monotonic clock in WSL with the OS file cache; they show relative cost, not disk latency.

Handoff for part 3 (index on roll-no)

- Use `SP_InsertRec`'s RID directly as the AM `recId`; `REC_SplitRid` recovers (page, slot).
- Extract the roll-no by splitting the stored line on ";" (confirm which field holds roll-no before indexing).
- "Index on existing file" = `SP_ScanOpen/SP_ScanNext` + `AM_InsertEntry`; "incremental" = insert record then index entry.
- If records are deleted, remove the index entry first (decision 2). The AM layer itself has not yet been rebuilt or tested.

Build, test and reproduce (Linux/WSL, repository root)

```
make -C toydb/splayer                                  # also rebuilds PF with -std=gnu17 -fpermissive
python3 toydb/splayer/run_tests.py --with-pf           # 26 cases, report in outputs/slotted-pages/
python3 -m venv ~/.venvs/toydb && ~/.venvs/toydb/bin/pip install matplotlib
~/.venvs/toydb/bin/python toydb/splayer/run_experiments.py   # settings: toydb/splayer/experiments.json
```

ToyDB: index construction summary (1)

Part 3, Part A: Methods 1 and 2 | 2 November 2026

Part 3 builds a B+ tree index on the Student file, keyed on roll-no, using the supplied AM layer. Three construction methods are to be compared. This document covers what is finished so far: **Method 1** (index an existing file), **Method 2** (build the file and the index together), the shared benchmark and validation code, the test results and the preliminary measurements. **Method 3** (bulk loading from a pre-sorted file) is Part B and has not been started.

> **Status.** Part A complete: both methods build valid indexes that the normal AM search APIs can read. All 4 test runs pass, and the core tests also pass under AddressSanitizer and UBSan. Preliminary results are below. Part B (Method 3) is pending and plugs into the existing benchmark without changing it.

1. What was built

File (toydb/amindex/)	Purpose
`ixbuild.c`	Method 1, Method 2, and the method table `IX_Methods[]` where Method 3 is registered
`ixindex.h/.c`	Shared code: roll-no key extraction, measurement (time + PF counters), query sets and runner, tree inspection, validation
`amwrap.h`	Includes the K&R `am.h` safely from modern C and adds AM prototypes
`ixbench.c`	Benchmark CLI: builds with method K, checks the tree, validates, times the query file; one JSON line per run
`testindex.c`	Correctness tests (real data and a synthetic 150,000-record file)
`run_tests.py`, `run_experiments.py`, `experiments.json`	Test runner and configurable experiment runner; results go to `outputs/index-construction/`
`README.md`	Build/run instructions, index file format and the Part B handover

Reused without changes: the PF layer (part 1) for paging and I/O counters, and the slotted-page Student file (part 2) for records and RIDs. The only existing file edited is `toydb/amlayer/amscan.c` (three bug fixes, section 5).

2. The two methods

Method	Steps	What "construction" measures
1. Existing file	Open the existing Student file; `AM_CreateIndex`; sequential scan (`SP_ScanNext`); for each record extract roll-no and RID and call `AM_InsertEntry`; close both files.	Create + scan + inserts + closing (flushes dirty pages). Writing the file first and opening it are reported as setup.
2. Incremental	Create an empty Student file and an empty index; for each dataset record call `SP_InsertRec` (gives the RID), then `AM_InsertEntry`; close both files.	Everything: the file and the index are built together, so there is no separate setup.

3. Key design decisions

- **Roll-no** is field 2 of each line (e.g. `95302001`, `96D07001`). All 17,813 values are 8 characters and some contain letters, so the key is an AM `'c'` key of length 8, compared in byte order.
- **RID** is part 2's `page × 65536 + slot`, a non-negative `int`, which is exactly AM's `recId` type.
- **Duplicates:** the data has 3 roll-nos that appear twice. AM stores one key with a list of RIDs, and an EQUAL search returns all of them (17,813 entries, 17,810 distinct keys).
- **Records without a roll-no** are stored but not indexed and are counted as skipped (none in the real data).
- **Index file** is the unmodified AM format: PF file `<studentFile>.0`, opened with `PF_OpenFile`.

Full options and reasoning: `outputs/design-decisions/part3-design-decisions.md`.

Preliminary results

Dataset `data/student.txt` (17,813 records). 2,500 point queries: 2,000 existing roll-nos and 500 absent ones, generated once with seed 42 and run in the same order for every method. LRU replacement. Each value is the median of 5 independent runs; query time is the median of 5 repetitions per run, each starting with none of the index's pages in the buffer pool.

Pages accessed = PF logical reads (page requests to the buffer pool). **Disk reads/writes** = PF physical I/O. Construction and query numbers come from separate counter snapshots.

Required comparison table (20-frame buffer pool)

Metric	Method 1 (existing)	Method 2 (incremental)
Records indexed	17,813	17,813
Construction time	**9.56 ms**	**17.18 ms**
Setup outside construction	8.01 ms	0.00 ms
Setup + construction (end to end)	17.58 ms	17.18 ms
Construction pages accessed	35,931	52,838
Construction disk reads / writes	519 / 176	77 / 640
Query completion time (2,500 queries)	**3.23 ms**	**3.09 ms**
Query pages accessed	**9,522**	**9,522**
Pages accessed per query	3.81	3.81
Query disk reads	2,060	2,060
Tree height / internal / leaf pages	2 / 1 / 109	2 / 1 / 109
Mean leaf fill	64.4%	64.4%

Effect of buffer pool size

Buffer frames	M1 construction	M2 construction	M1 constr. disk R / W	M2 constr. disk R / W	Query time M1 / M2	Query disk reads
20	9.56 ms	17.18 ms	519 / 176	77 / 640	3.23 / 3.09 ms	2,060
50	10.39 ms	15.59 ms	482 / 139	39 / 602	2.44 / 2.45 ms	1,375
200	9.41 ms	16.32 ms	463 / 120	15 / 578	1.25 / 1.29 ms	110

Pages accessed do not depend on the pool size (35,931 / 52,838 for construction, 9,522 for queries). With 200 frames the whole 110-page index fits in the pool, so each query page is read from disk only once.

What the numbers show

- **Methods 1 and 2 build the same tree.** Method 1's sequential scan returns records in RID order, which is the order Method 2 inserted them, so AM receives the same key sequence. Query cost is therefore identical, and the methods differ only in construction cost.
- **Construction alone:** Method 1 is about 45% faster and makes about 32% fewer page requests, because the Student file already exists. **End to end** (including writing the file) the two cost about the same.
- **I/O pattern:** Method 1 re-reads the whole Student file (about 464 pages) during its scan. Method 2 reads almost nothing from disk but writes more, because Student-file and index pages compete for the same buffer pool.
- **Per query:** about 4 page requests for a hit (root lookup for the leftmost leaf, root, leaf, leaf again in `AM_FindNextEntry`) and about 3 for a miss.
- Leaves are only 64% full because inserts split leaves in half. A sorted bulk load (Method 3) should give fuller leaves and fewer pages; that is the main comparison for Part B.

Correctness and AM fixes

4. Tests (all passing)

Check	How
Existing, non-existing and several roll-nos	Explicit searches after reopening the index, including the 3 duplicated roll-nos
Every RID points to the right Student record	`IX_Validate`: for **every** distinct key, the EQUAL scan returns exactly as many distinct RIDs as the dataset has records with that roll-no, and each RID's record has that roll-no; plus 202 absent keys below, between and above the real ones
Entry count = records indexed	Full AM scan (17,813 entries in key order) and a walk of the whole tree
Close and reopen	Validation and queries reopen the index; queries are repeated on a second reopen and must give identical answers and page counts
B+ tree invariants	`IX_InspectTree`: page types, sorted keys, separator ranges, equal leaf depth, complete leaf chain, leftmost leaf = page 2
Index valid during Method 2	Invariants and searches checked at 20 points while records are still being inserted
Both methods agree	Identical answers (RID checksum) for 600 queries
Node splitting and multiple levels	Synthetic 150,000-record file (random order, duplicates, records without a roll-no) gives a 3-level tree: 5 internal pages, 894 leaves
No page left fixed	Tests run with 3, 20 and 200 buffer frames; 3 is AM's minimum (a root split fixes 3 pages)

Log: `outputs/index-construction/test-results.txt`. Run with `python3 toydb/amindex/run_tests.py`.

5. Bugs found and fixed in the supplied AM layer

Bug	Effect	Fix (in `amscan.c`)
`AM_OpenIndexScan` pushes the search path onto a 50-entry stack that only insert/delete empty	Once the tree has 2+ levels, the 51st search writes past the end of `AM_Stack` (found with AddressSanitizer)	FIX 1: empty the stack after the search
Absent key that sorts after the last key of a leaf: the next leaf's header overwrites the current one before its page number is used	The wrong page is unfixed, so the search fails with `AME_PF` and the next leaf stays fixed in the buffer	FIX 2: save the next leaf's page number first
Same pattern in `AM_FindNextEntry` for `NOT_EQUAL` scans	Same as above	FIX 3

Each fix is a few lines and is marked `PART 3 FIX n`; search results are otherwise unchanged. Separately, `toydb/amlayer/pf.h` still says pages are 1020 bytes while the PF layer uses 4096. That file was not edited: the amindex Makefile compiles copies of the AM sources so they use the correct header, and a test checks that the root's `maxKeys` is 340 (the 4096-byte value).

6. How to build and run

```
make -C toydb/amindex                     # builds ixbench and testindex
python3 toydb/amindex/run_tests.py        # all tests  -> outputs/index-construction/test-results.txt
python3 toydb/amindex/run_experiments.py  # benchmark  -> outputs/index-construction/comparison.md

# one method by hand
./ixbench queries --data ../../data/student.txt --hits 2000 --misses 500 --seed 42 --out work/q.txt
./ixbench run --method 1 --data ../../data/student.txt --queries work/q.txt --workdir work --validate
```

Build under Linux/WSL. Generated Student and index files go to a temporary or `--workdir` directory and are deleted after each run; `data/student.txt` is only read.

Handover for Part B and remaining work

7. Adding Method 3 (bulk loading)

1. Write `ixbulk.c` with `int IX_Method3(const IX_BuildSpec *spec, IX_BuildResult *r)`: sort a copy of the dataset by key, write the sorted Student file (measured as setup), bulk-build the index from the sorted (key, RID) pairs (measured as construction, including closing the files), and set the record counts.
2. Register it in `IX_Methods[]` (`ixbuild.c`) and add `build/ixbulk.o` to the Makefile.
3. Check it with `ixbench run --method 3 --validate`, add `3` to `"methods"` in `experiments.json` and rerun `run_experiments.py`. The table gains a Method 3 column with the same queries and the same procedure.

8. Index format rules Method 3 must keep

Rule	Reason
Root is always page 0	AM searches start at the file's first page, and inserts detect a root split by page number 0
Leftmost leaf is page 2 when the root is internal	Hard-coded in `GetLeftPageNum` for full and `<`, `<=`, `!=` scans; checked by `IX_InspectTree`
Leaf: 24-byte header, then sorted `[8-byte key][2-byte list offset]`; RID nodes `[4-byte RID][2-byte next]` grow down from the page end	At most 254 distinct keys per leaf
Internal: 8-byte header, then `[child0][key1][child1]...`, `maxKeys = 340`	Child i holds keys from key_i (inclusive) up to key_{i+1}; later AM inserts must still split correctly
All RIDs of one key stay in one leaf	AM's EQUAL scan stops at the end of the leaf

Method 3's Student file is sorted, so its RIDs (and RID checksum) will differ from Methods 1 and 2; correctness is shown by `--validate` and by equal found/RID counts. Full details: `toydb/amindex/README.md`.

9. Remaining work and limitations

- **Pending:** Method 3 (bulk loading), its comparison against Methods 1 and 2, and the final Part 3 summary.
- Only point queries are benchmarked; range queries are not measured.
- AM allows about 670 RIDs per key (the real data has at most 2).
- Times are in milliseconds and vary between runs, so medians are reported; page counts are deterministic and are the more reliable metric.
- The supplied AM test programs (`amlayer/test*.c`) have not been rebuilt.

10. Files

Location	Content
`toydb/amindex/`	All Part A code, tests and runners
`toydb/amlayer/amscan.c`	Three marked bug fixes
`outputs/index-construction/`	`comparison.md`, `summary.csv`, `results.csv`, `raw_results.jsonl`, `queries.txt`, `config.json`, test logs, this summary's source
`outputs/design-decisions/part3-design-decisions.md`	Design decisions with options and evidence

Part 3B: sorted bulk loading

Method 3 and final three-method integration | 3 November 2026

1. Sorting and correct record identifiers

ixbulk.c sorts a separate array of original record ordinals and 8-byte roll-number keys. Equal keys retain original order. Raw record bytes and the original dataset are unchanged. Missing or oversized keys are stored last and skipped by the index. Setup writes a reordered slotted Student file and collects the actual new RID returned by each insertion.

The resulting in-memory (key, RID) stream is sorted. First-fit record placement can fill space on earlier pages, so the Student file's physical scan order is not guaranteed to be sorted. The sorted stream is the bulk loader's input; it always references the correct new records.

2. Bottom-up construction in the existing AM format

Equal keys are grouped and all their RIDs remain in one leaf. Leaves are packed by byte capacity, with the final pair balanced when possible. Key-directory entries and RID linked lists use the unchanged AM layout. Parent levels are built from child page numbers and first-key separators; balanced fanout avoids a singleton last internal node. The root is written at page 0.

For internal-root trees, the first leaf is page 2 to preserve the supplied AM scan convention. Page 1 is used as the last leaf, so every allocated page is reachable. Empty and single-leaf trees occupy root page 0. The maximum internal key count remains 340. No alternate search engine or index format was introduced.

3. Validation and compatibility

Five groups pass: bulk boundaries and AM mutations; real/synthetic data with 3 and 20 frames; real data with 200 frames; and a validated three-method benchmark. The synthetic integration dataset has 150,000 records. Bulk edge cases include 0, 1, 254, 255, 508, 509 and 87,000 records, including a three-level bulk tree. Tests exercise MRU, duplicate-list limits, byte-for-byte preservation, normal AM insert/delete, scanning, absent keys and reopening.

The shared validator checks every distinct key and its RIDs against Student records, full AM scans, leaf links and separator ranges. Method 3's RIDs differ from Methods 1-2, so result counts and independent record validation are compared instead of cross-method RID checksums.

4. Limits and deferred work

Sorting and working arrays use O(n) RAM, not external merge sorting. A duplicate group must fit on one leaf: 677 RIDs fit for a single-key page; 678 are explicitly rejected. Heavy duplicate skew can also expose limitations in later legacy AM splits. Only point queries are benchmarked. PF/AM compiler warnings remain; local replacement is deferred. PF, SP and AM implementation source was not changed for Part 3B.

Part 3B: final results and reproduction

Dataset: 17,813 records, 17,810 distinct roll numbers. The same 2,500 queries (2,000 hits and 500 misses) run in identical order. Five independent runs per method/pool size, five cold ToyDB query repetitions per run: 45 builds. Tables report medians. OS caching may still apply.

20-frame pool	Method 1	Method 2	Method 3
Construction time (ms)	16.96	33.08	1.50
Setup + construction (ms)	36.45	33.08	35.68
Construction logical reads	35,931	52,838	72
Query time (ms)	6.10	6.06	7.31
Query logical reads	9,522	9,522	9,511
Query physical reads	2,060	2,060	1,843
Leaf pages	109	109	71

Method 3 has 98.6% mean leaf utilization versus 64.4% for Methods 1-2, with 71 rather than 109 leaves. All three return 2,000 hits, 500 misses and 2,002 RIDs (including duplicate-key results).

Interpretation and fair timing boundaries

Bulk loading avoids repeated insertion searches and split propagation. Sorting and reordered-file creation are measured as setup; construction consumes the prepared sorted stream and includes final close/flush. Compare setup plus construction for the full cost: bulk is not the fastest at 20 frames in this run, but is fastest at 50 and 200 frames. Query times also vary; fuller leaves do not guarantee faster searches. Physical I/O means full-page OS transfers, not hardware disk activity. Fresh allocations themselves do not count as logical reads.

Build and run from the repository root

```
make -C toydb/amindex
python3 toydb/amindex/run_tests.py
python3 toydb/amindex/run_experiments.py
python3 package_submission.py
```

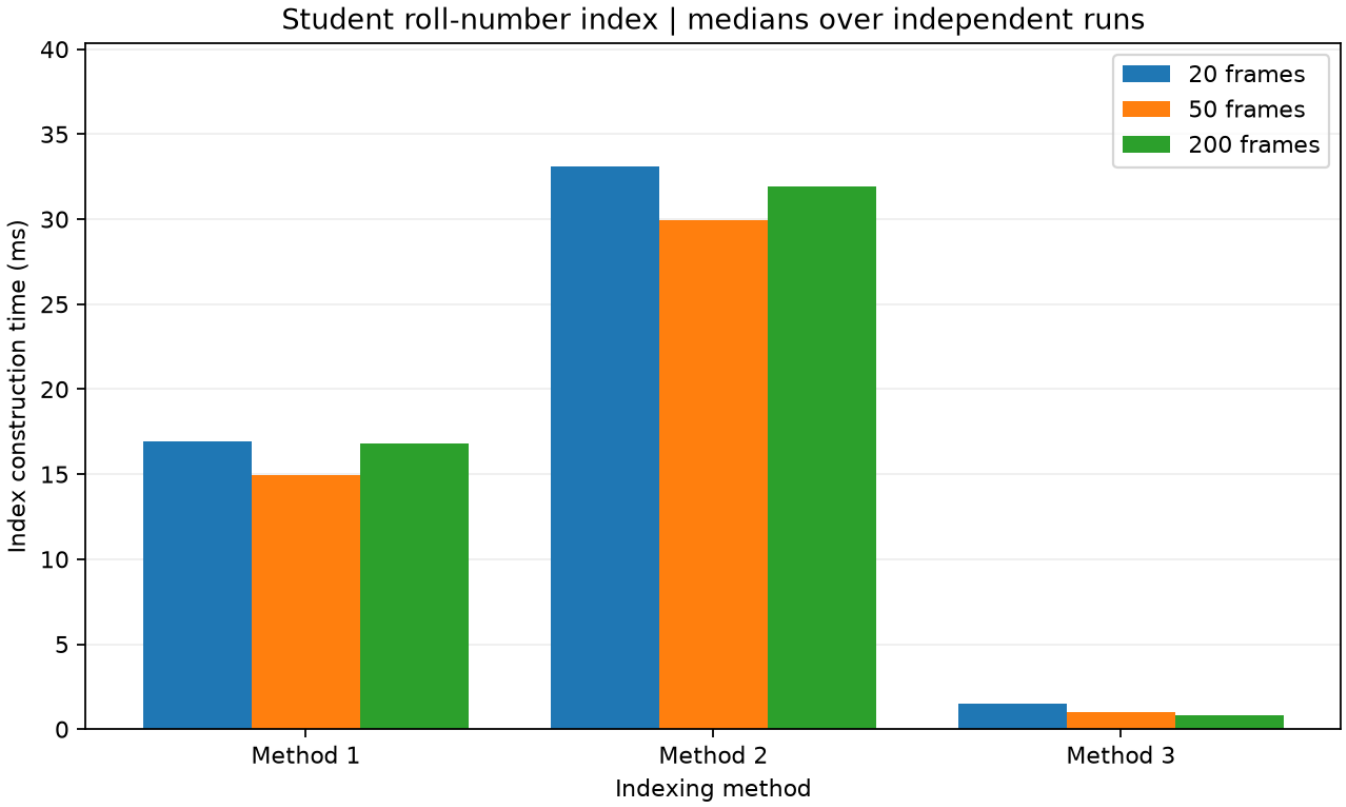
Graph generation requires matplotlib. Configure methods, buffers, query counts, seed and repetitions in toydb/amindex/experiments.json. To run one method, use ixbench run --method 1, 2 or 3 with --data, --queries, --workdir and --validate; complete commands are in toydb/amindex/README.md. The archive was tested after clean extraction.

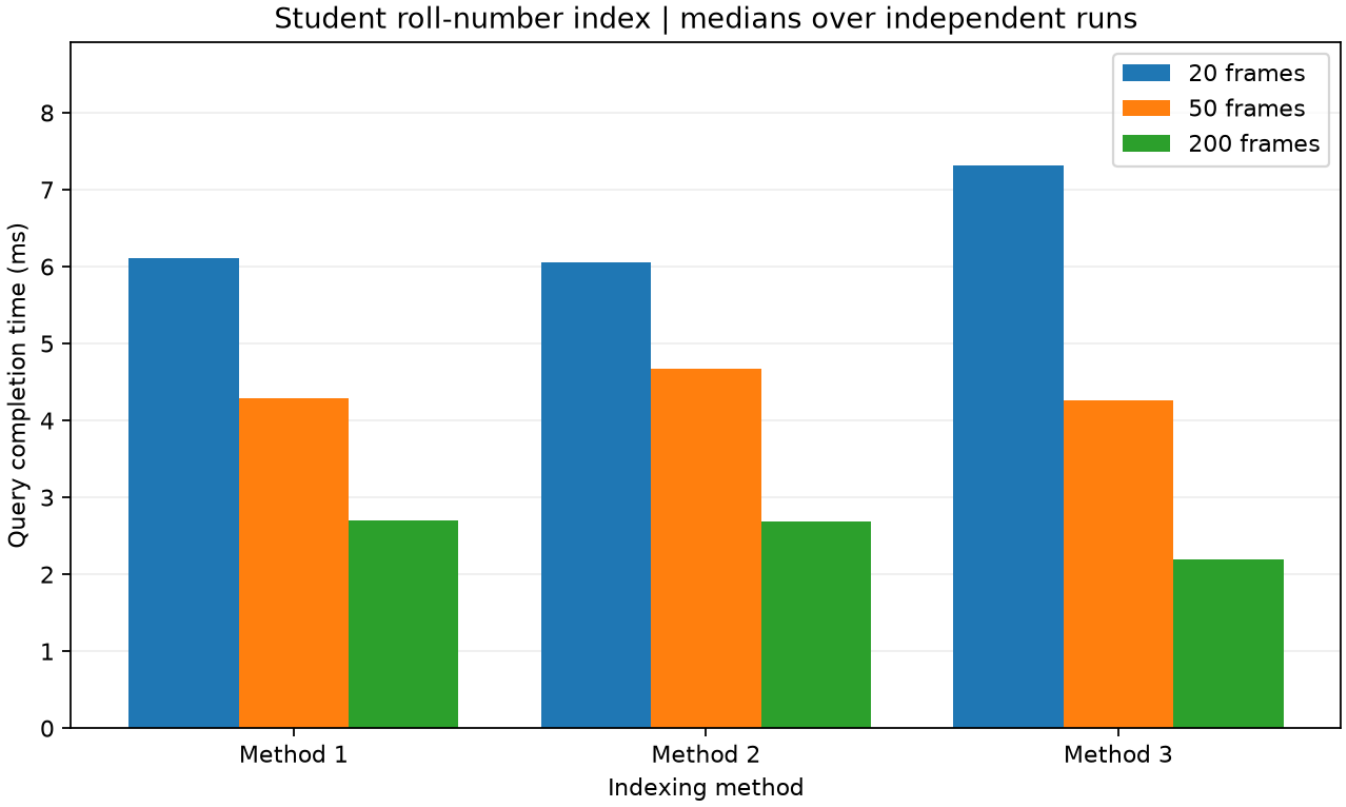
Files and evidence

New: ixbulk.c, testbulk.c (toydb/amindex/) and package_submission.py (root). Modified: ixbuild.c, ixindex.h, Makefile, testindex.c, run_tests.py, run_experiments.py, experiments.json, root/amindex READMEs and the Part 3 decision log. Results: outputs/index-construction/ contains CSV/JSON measurements, comparison.md, analysis.md, test/sanitizer logs, archive verification and four graphs. Earlier Part 3A figures remain historical.

Part 3B: construction and query time

Same dataset and query file across methods. Bars show medians over five independent runs; colors identify buffer-pool capacities.





Part 3B: page-access comparisons

Same dataset and query file across methods. Bars show medians over five independent runs; colors identify buffer-pool capacities.

